UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT

Ms. Pac-Man Game using Java Swing

Program Name: BCA
Subject Name/Code: Java Programming Lab
(23CAP-352)

Submitted by:

Name: Chandan Kartik
UID: 23BCA10383
Section: 23BCA-(8)B

Submitted to:

Name: Divij Sumbly
Designation: Assistant Professor

Project Report: Ms. Pac-Man Game using Java Swing

1. **Title: -** Ms. Pac-Man Game in Java Swing

The aim of this project is to design and develop a Ms. Pac-Man Game game using Java Swing - A 2D arcade-style game inspired by the classic Ms. Pac-Man, built entirely with Java's Swing and AWT libraries. The player navigates a maze, collects pellets to earn points, and must avoid a roaming ghost — demonstrating core concepts of GUI programming, game loops, event-driven design, and 2D rendering. That provides a graphical user interface allowing players to play on the computer. The game opens a dedicated window, provides a clean and interactive visual experience.

2. Objective: -

The objective of this project is to design and develop a simple Ms. Pac-Man game using Java. The game demonstrates concepts of GUI programming, event handling, and basic game logic using Java Swing.

The primary objectives of this project are:

- To apply Java Swing and AWT for building an interactive graphical desktop application.
- To implement a real-time game loop using javax.swing.Timer for continuous screen updates.
- To handle keyboard events (arrow keys) for smooth player movement.
- To design a tile-based map system that models walls, paths, and collectible pellets.
- To implement basic AI for ghost movement using random directional logic.
- To demonstrate collision detection between the player, walls, and the ghost entity.

3. Tools and Technologies Used:-

The following tools and technologies were used to develop this project:

- Programming Language: Java
- GUI Library: Swing (javax.swing)
- IDE: VS Code / IntelliJ IDEA / Eclipse
- Libraries Used:
    - java.awt (Graphics & UI)
    - java.awt.event (Event Handling)
    - javax.swing.Timer (Game loop)

- Java SE (JDK 8 or higher) – Core programming language used for all game logic
- Java Swing – GUI framework used for building the game window, board, buttons, and labels (JFrame, JPanel, JButton, JLabel)
- Java AWT (Abstract Window Toolkit) – Used for Graphics2D, Color, Font, BasicStroke, and custom painting via paintComponent()
- IDE: IntelliJ IDEA / Eclipse / Notepad++ with javac compiler – Used for writing, compiling, and running the Java source file

Category	Tool / Technology	Purpose
Programming Language	Java (JDK 8 or above)	Core language for game logic and GUI
GUI Framework	javax.swing.JPanel, JFrame, JOptionPanel	Window, game panel, and dialog rendering
Event Handling	java.awt.event.ActionListener, KeyListener	Game loop ticks and keyboard input
Graphics API	java.awt.Graphics, Color, Font	Drawing tiles, characters, and score text
Randomization	java.util.Random	Ghost AI random movement decisions
Build / Run	javac + java (CLI) or any Java IDE	Compiling and executing the game
IDE (Recommended)	IntelliJ IDEA / Eclipse	Code editing, debugging, and execution

4. System Requirements:-

- **Hardware:**
  - Processor: Minimum 1 GHz.
  - RAM: 2GB minimum.
  - Storage: At least 200 MB free disk space (for JDK and project files)
- **Software:**
  - Java Runtime Environment (JRE).
  - Any OS supporting Java (Windows, macOS, Linux).
  - Java Development Kit (JDK): Version 8 or higher

5. Algorithm / Logic:-

The Ms Pan-Man game follows a structured step-by-step process:

1. Game Initialization

- Create a game window using JFrame
- Initialize a 2D array (map[][]) for maze
- Set player and ghost starting positions

2. Map Creation

- Walls (1) and paths (0) are generated using conditions
- Boundary walls are fixed
- Internal walls are created using pattern logic

3. Game Loop (Timer)

- Runs every 120 ms using Timer
- Continuously updates:
  - Player movement
  - Ghost movement
  - Collision detection

4. Player Movement

- Controlled by arrow keys
- Moves only if next position is not a wall
- Score increases on movement

5. Ghost Movement

- Moves randomly using Random
- Chooses direction from {-1, 0, 1}

6. Collision Detection

- If player and ghost positions match:
    - Game stops
    - "Game Over" message shown

7. Rendering (Graphics)

- Draw maze (blue walls)
- Draw food (white dots)
- Draw player (yellow circle)
- Draw ghost (red circle)
- Display score

6. Procedure:-

1. **Initialization:** Create the JFrame and the JPanel (MsPacmanGame).
2. **Map Design:** Generate the boundaries and internal obstacles using nested loops in initMap().
3. **Event Handling:** Set up a javax.swing.Timer to refresh the game state every 120ms.
4. **Input Mapping:** Bind the arrow keys to change the player's directional velocity.
5. **Game Loop:** * Update player/ghost positions.
    - Check for collisions.
    - Call repaint() to draw the updated frame.
6. **End State:** Display a JOptionPane dialog when the player is caught.

7. Code:-

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.Random;

public class MsPacmanGame extends JPanel implements ActionListener, KeyListener
{

    Timer timer;
```

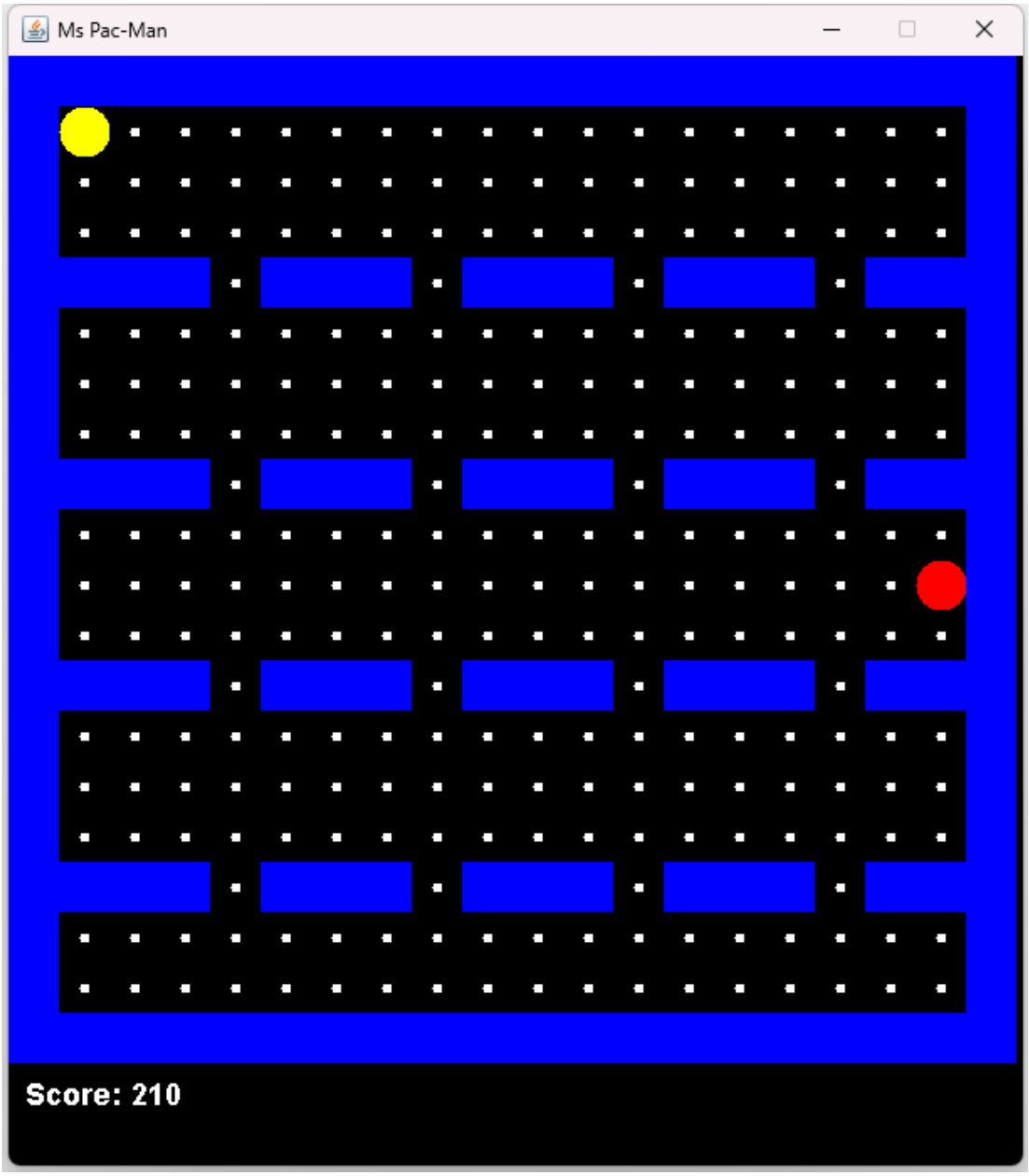
```
    int tile = 30;
    int rows = 20, cols = 20;

    int playerX = 1, playerY = 1;
    int dx = 0, dy = 0;

    int ghostX = 10, ghostY = 10;

    int score = 0;

    int[][] map = new int[rows][cols];

    public MsPacmanGame() {
        setFocusable(true);
        addKeyListener(this);

        initMap();

        timer = new Timer(120, this);
        timer.start();
    }

    void initMap() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                if (i == 0 || j == 0 || i == rows - 1 || j == cols - 1 || (i % 4 == 0 && j % 4 !=
0)) {
                    map[i][j] = 1; // wall
                } else {
                    map[i][j] = 0; // path
                }
            }
        }
    }

    public void paintComponent(Graphics g) {
        super.paintComponent(g);

        // Background
        g.setColor(Color.BLACK);
        g.fillRect(0, 0, getWidth(), getHeight());

        // Draw map
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
```

```
            if (map[i][j] == 1) {
                g.setColor(Color.BLUE);
                g.fillRect(j * tile, i * tile, tile, tile);
            } else {
                g.setColor(Color.WHITE);
                g.fillOval(j * tile + 12, i * tile + 12, 6, 6);
            }
        }
    }

    // Player (Ms Pac-Man)
    g.setColor(Color.YELLOW);
    g.fillOval(playerY * tile, playerX * tile, tile, tile);

    // Ghost
    g.setColor(Color.RED);
    g.fillOval(ghostY * tile, ghostX * tile, tile, tile);

    // Score UI
    g.setColor(Color.WHITE);
    g.setFont(new Font("Arial", Font.BOLD, 18));
    g.drawString("Score: " + score, 10, rows * tile + 25);
}

public void actionPerformed(ActionEvent e) {
    movePlayer();
    moveGhost();

    if (playerX == ghostX && playerY == ghostY) {
        timer.stop();
        JOptionPane.showMessageDialog(this, "Game Over! Score: " + score);
    }

    repaint();
}

void movePlayer() {
    int nx = playerX + dx;
    int ny = playerY + dy;

    if (map[nx][ny] == 0) {
        playerX = nx;
        playerY = ny;
        score++;
    }
```

```
    }

    void moveGhost() {
        Random r = new Random();
        int[] d = {-1, 0, 1};

        int nx = ghostX + d[r.nextInt(3)];
        int ny = ghostY + d[r.nextInt(3)];

        if (nx >= 0 && ny >= 0 && nx < rows && ny < cols && map[nx][ny] == 0) {
            ghostX = nx;
            ghostY = ny;
        }
    }

    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_UP) { dx = -1; dy = 0; }
        if (e.getKeyCode() == KeyEvent.VK_DOWN) { dx = 1; dy = 0; }
        if (e.getKeyCode() == KeyEvent.VK_LEFT) { dx = 0; dy = -1; }
        if (e.getKeyCode() == KeyEvent.VK_RIGHT) { dx = 0; dy = 1; }
    }

    public void keyReleased(KeyEvent e) {}
    public void keyTyped(KeyEvent e) {}

    public static void main(String[] args) {
        JFrame frame = new JFrame("Ms Pac-Man");

        MsPacmanGame game = new MsPacmanGame();
        frame.add(game);

        frame.setSize(620, 700);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setResizable(false);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

9.Output: RUN

```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

D:\java game ------------------------------------------\New folder>dir
 Volume in drive D is New Volume
 Volume Serial Number is 1E45-1AD1

 Directory of D:\java game ------------------------------------------\New folder

16-04-2026  07:52    <DIR>          .
16-04-2026  07:49    <DIR>          ..
16-04-2026  07:52             4,087 MsPacmanGame.class
16-04-2026  07:50             3,717 MsPacmanGame.java
               2 File(s)          7,804 bytes
               2 Dir(s)  186,249,744,384 bytes free

D:\java game ------------------------------------------\New folder>java MsPacmanGame
```



10. links:-

https://github.com/chndnkrtik/MsPac-Man-Game-

11. Conclusion:-

The Ms. Pac-Man game developed using Java Swing successfully demonstrates the implementation of fundamental concepts of GUI-based application development and basic game design. The project integrates key programming techniques such as event handling, object-oriented design, and real-time updates using a timer. Through this project, a functional and interactive game environment was created where the player can control movement, interact with the game map, and respond to dynamic elements like the ghost. The use of Java Swing provided a simple yet effective way to build graphical components and render game visuals.

Overall, this project enhances practical knowledge of Java programming and GUI development while encouraging further improvements such as adding sound effects, animations, and advanced AI. It serves as a valuable learning experience in combining programming logic with user interface design to create an interactive application.

12. Learning Outcomes:-

- **Graphics Programming:** Understanding the coordinate system in Java AWT where starts at the top-left corner.
- **Concurrency:** Managing a real-time game loop using a Timer thread.
- **Collision Logic:** Mastering grid-based boundary checking to prevent characters from "walking through walls."
- **Event-Driven Programming:** Implementing interfaces (KeyListener, ActionListener) to respond to user actions and time intervals.

Through this project, we learned:

- How to create GUI applications using Java Swing
- Handling keyboard input using KeyListener
- Implementing a game loop using Timer
- Working with 2D arrays for map design
- Basic game development concepts (movement, collision, scoring)

13. References:-

- TutorialsPoint – Java Swing Tutorial
- GeeksforGeeks – Java AWT and Swing Guide
- YouTube – Java GUI Programming Tutorials
- Javatpoint – Java Swing Components Reference